

FinShare — One Backend Addition Needed

Note for Abhilash · 03 April 2026

Hi Abhilash, we are now building the Login, Register, and Forgot Password screens on the frontend. While reviewing the backend, we confirmed that Login and Register are already there and working perfectly. There is just one missing endpoint needed for the Forgot Password flow.

What is Already Done — No Action Needed

POST /api/Auth/login

Confirmed in AuthController.cs. Works with { userName, password }.

Returns JWT token + memberId + email. All good.

POST /api/Auth/register

Confirmed in AuthController.cs. Uses sp_RegisterUser.

Takes { memberName, userName, emailAddress, mobile?, password }.

Rejects duplicate emails automatically. All good.

What Is Needed — One New Endpoint

The Forgot Password feature requires a way for a user to recover access to their account by email. The frontend will send the user's email address to the backend, and the backend should send a password reset link to that email.

Please add two endpoints:

Endpoint 1 — Request a password reset

POST /api/Auth/forgot-password

Request body: { email: string }

What it does: Generate a secure reset token, store it against the user record

(with an expiry — e.g. 1 hour), then send an email to the user

with a reset link: <https://finshare.me/reset-password?token=XXXX>

Response: { success: true, message: 'Reset link sent to your email' }

(Always return success — do not confirm whether the email exists, as this prevents email enumeration attacks.)

Endpoint 2 — Submit the new password

POST /api/Auth/reset-password

Request body: { token: string, newPassword: string }

What it does: Validate the token (check it exists and has not expired),

then update the user's password hash using BCrypt.

Invalidate the token after use so it cannot be reused.

Response: { success: true, message: 'Password updated successfully' }

On error: { success: false, message: 'Reset link has expired or is invalid' }

Suggested Database Change

You will need a way to store the reset token against a member. The simplest approach is to add two columns to the existing Users table:

```
ALTER TABLE Users
ADD PasswordResetToken NVARCHAR(200) NULL,
    PasswordResetExpiry DATETIME NULL;
```

Then add two stored procedures — one to save the token (sp_SavePasswordResetToken) and one to validate and clear it (sp_ResetPassword). This keeps it consistent with the existing Dapper + SP pattern used throughout the project.

Note on email sending:

The reset link needs to be sent by email. If the project does not yet have an email service configured, you can use any of these options:

- SendGrid (free tier is sufficient — 100 emails/day)
- SMTP via Gmail or Outlook
- Azure Communication Services (if already on Azure)

Let me know which you prefer and we can help with the integration.

Thank you as always — the backend is really clean and well-structured.